

# NFL Big Data Bowl 2026 - Player Movement Prediction

## Kaggle Machine Learning Project Report

Alae El Fahsi  
Muhammad Ammad Ismail  
Tanjim Hossain  
Kiflom Berhe  
Amudhaanandheswaran Selvaraj

December 12, 2025

### Abstract

In this project we worked on the Kaggle competition *NFL Big Data Bowl 2026 - Prediction*. The goal is to predict player positions while the ball is in the air after a forward pass, using tracking data from NFL Next Gen Stats.

We describe how we approached the task: understanding the data, engineering features that capture movement and game context, and building supervised models. Several models were tried (XGBoost, LightGBM, CatBoost, LSTM), but our final solution is an XGBoost model that predicts player displacement using history and geometric features, trained with time-based cross-validation by week. This model reached a public Kaggle score of 0.773 with a rank of 790 on the public leaderboard and clearly outperformed our baselines.

# Contents

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                | <b>3</b>  |
| <b>2</b> | <b>Data and Problem Setup</b>                      | <b>3</b>  |
| 2.1      | Data Structure . . . . .                           | 3         |
| 2.2      | Prediction Target and Metric . . . . .             | 3         |
| <b>3</b> | <b>Feature Engineering</b>                         | <b>4</b>  |
| 3.1      | Play Direction Normalization . . . . .             | 4         |
| 3.2      | Throw Frame and History Features . . . . .         | 4         |
| 3.3      | Kinematic and Ball-Relative Features . . . . .     | 4         |
| 3.4      | Player-Level Features and Encoding . . . . .       | 5         |
| <b>4</b> | <b>XGBoost Model (Main Model)</b>                  | <b>5</b>  |
| 4.1      | Model Design . . . . .                             | 5         |
| 4.2      | Time-Based Cross-Validation and Ensemble . . . . . | 6         |
| 4.3      | Kaggle Inference Setup . . . . .                   | 6         |
| 4.4      | Performance and Leaderboard Result . . . . .       | 6         |
| <b>5</b> | <b>Other Tested Models</b>                         | <b>7</b>  |
| 5.1      | LightGBM Models . . . . .                          | 7         |
| 5.2      | CatBoost Model . . . . .                           | 7         |
| 5.3      | LSTM Sequence Model . . . . .                      | 8         |
| 5.4      | Summary of Model Performance . . . . .             | 8         |
| <b>6</b> | <b>Discussion</b>                                  | <b>9</b>  |
| <b>7</b> | <b>Conclusion</b>                                  | <b>9</b>  |
| <b>8</b> | <b>Acknowledgement</b>                             | <b>10</b> |
|          | <b>Appendix</b>                                    | <b>10</b> |

# 1 Introduction

The NFL Big Data Bowl 2026 competition asks participants to predict how NFL players move after the quarterback throws a forward pass. The data comes from tracking sensors and is sampled at 10 frames per second. For each play, we observe all players before the ball is thrown and need to predict their future positions while the ball is in the air.

Each row in the evaluation set is identified by:

- **game\_id**, **play\_id**: which game and play,
- **nfl\_id**: which player,
- **frame\_id**: which frame after the throw.

The competition provides training input files (tracking data before the throw) and training output files (true player positions after the throw). A flag **player\_to\_predict** indicates which players count for the score.

The evaluation metric is Root Mean Squared Error (RMSE) on the predicted  $(x, y)$  coordinates on the field. The leaderboard is computed on games that we do not see during training, so it is important to use a validation strategy that respects time and does not mix early and late games.

Our main goal in this project is not only to reach a good leaderboard score, but also to follow a clear and correct machine learning pipeline: first understand the data and define the prediction task, then engineer useful features from the raw tracking data, then choose and train models with a fair validation setup, and finally compare models and select the best one.

## 2 Data and Problem Setup

### 2.1 Data Structure

The training data covers the 2023 NFL season and is split into weeks. For each week  $w$ , we have:

- **input\_2023\_wXX.csv**: tracking data up to and including the frame where the ball is thrown,
- **output\_2023\_wXX.csv**: true future positions for each scored player while the ball is in the air.

The tracking data contains for each player and frame:

- spatial information:  $x$ ,  $y$ ,
- movement information: speed  $s$ , acceleration  $a$ , direction **dir**, orientation  $\phi$ ,
- context: play direction (left or right), ball landing position, number of frames to predict,
- player information: position, role, side (offense/defense), height, weight, date of birth.

The output data contains the same identifiers and gives  $x_{\text{future}}$  and  $y_{\text{future}}$  for each frame while the ball is in the air. We filter the data to only keep rows where **player\_to\_predict = True**, because only these players are scored in the competition.

### 2.2 Prediction Target and Metric

Instead of predicting the absolute coordinates directly, we predict the displacement relative to the throw frame:

$$\Delta x_{\text{future}} = x_{\text{future}} - x_{\text{throw}}, \quad \Delta y_{\text{future}} = y_{\text{future}} - y_{\text{throw}}.$$

Here  $(x_{\text{throw}}, y_{\text{throw}})$  are the coordinates of the player in the last frame before the throw. This makes the problem easier, because the model focuses on how far the player moves, not where on the field the play started.

At inference time, we reconstruct the coordinates using:

$$x_{\text{pred}} = x_{\text{throw}} + \Delta x_{\text{pred}}, \quad y_{\text{pred}} = y_{\text{throw}} + \Delta y_{\text{pred}}.$$

The competition metric is RMSE on  $(x, y)$ :

$$\text{RMSE} = \sqrt{\mathbb{E}[(x_{\text{true}} - x_{\text{pred}})^2 + (y_{\text{true}} - y_{\text{pred}})^2]}.$$

We use the same metric both for validation and to interpret the Kaggle score.

### 3 Feature Engineering

A large part of our work was to design features that help the models understand how players are moving and how they react to the ball. In this section we summarize the main steps.

#### 3.1 Play Direction Normalization

In the raw data, some plays have the offense moving to the left, some to the right. To avoid learning two mirrored versions of the same patterns, we normalize all plays so the offense always moves from left to right.

For plays with `play_direction = left`, we:

- reflect  $x$  and  $y$ :  $x \leftarrow 120 - x$ ,  $y \leftarrow 53.3 - y$ ,
- reflect the ball landing coordinates in the same way,
- add  $180^\circ$  to the direction and orientation angles (modulo  $360^\circ$ ),
- flip history velocity components when present.

After this transformation, all plays are expressed in a common coordinate system, which makes it easier for the model to learn general rules.

#### 3.2 Throw Frame and History Features

For each player, we first find the last frame before the throw (the “throw frame”). We call the coordinates in this frame  $(x_{\text{throw}}, y_{\text{throw}})$ .

We then summarize the last few frames (for example, 5 frames) before the throw into history features:

- mean velocity in  $x$  and  $y$ : `mean_vx`, `mean_vy`,
- mean and maximum acceleration: `mean_a`, `max_a`,
- standard deviation of direction: `std_dir`,
- number of frames in the history window: `frames_hist`,
- approximate distance covered in the last frames: `dist_hist`.

These features summarize how the player has been moving just before the throw, which is a strong indicator of where they are heading.

#### 3.3 Kinematic and Ball-Relative Features

At the throw frame we compute:

- time since throw for each future frame:  $t = \text{frame\_id}/10$  seconds,
- direction in radians: `dir_rad`,
- velocity components at throw:  $v_{x0} = s \cos(\text{dir\_rad})$ ,  $v_{y0} = s \sin(\text{dir\_rad})$ .

We also compute features relative to the ball landing point:

- distance from throw position to ball landing:

$$\text{dist\_ball} = \sqrt{(\text{ball\_land\_x} - x_{\text{throw}})^2 + (\text{ball\_land\_y} - y_{\text{throw}})^2},$$

- angle from the throw position to the ball landing:  $\text{angle\_to\_ball} = \arctan 2(\Delta y, \Delta x)$ ,
- ball flight time:  $\text{time\_to\_land} = \text{num\_frames\_output} / 10$ ,
- required average speed to reach the ball:  $\text{req\_speed} = \text{dist\_ball} / \text{time\_to\_land}$ ,
- ratio between current speed and required speed:  $\text{speed\_ratio} = s / (\text{req\_speed} + 10^{-6})$ ,
- angular difference between current direction and direction to ball (wrapped to  $[-\pi, \pi]$ ):  $\text{angle\_diff}$ ,
- simple constant-velocity baseline:  $\text{dx\_cv} = \text{mean\_vx} \times \text{time\_to\_land}$ ,  $\text{dy\_cv} = \text{mean\_vy} \times \text{time\_to\_land}$ .

We also include distances to the sidelines and endzone at the throw:

- $\text{dist\_sideline\_bottom} = y_{\text{throw}}$ ,
- $\text{dist\_sideline\_top} = 53.3 - y_{\text{throw}}$ ,
- $\text{dist\_endzone} = 120 - x_{\text{throw}}$ .

These geometric features give the model information about where the ball is going and how much room the player has on the field.

### 3.4 Player-Level Features and Encoding

From the player information we derive:

- **height\_in**: height in inches, computed from strings like “6-1”,
- **weight\_lb**: weight in pounds,
- **age\_years**: approximate age in 2023,
- **abs\_yardline**: absolute yardline number as a numeric value.

Categorical variables like player role, position, and side are encoded with one-hot encoding. This produces binary columns that are easy for tree-based models to use.

## 4 XGBoost Model (Main Model)

### 4.1 Model Design

Our main model uses XGBoost, a gradient boosting method that works well for tabular data. We train two separate regressors, one for  $\Delta x_{\text{future}}$  and one for  $\Delta y_{\text{future}}$ . Both models share the same feature set, which includes:

- time and kinematics at the throw,
- history features over the last frames before the throw,
- ball and field geometry features,
- player-level numeric features,
- one-hot encoded player role, position, and side.

We use `XGBRegressor` with the following main hyperparameters:

- `n_estimators` = 600,
- `learning_rate` = 0.05,
- `max_depth` = 7,
- `subsample` = 0.9,
- `colsample_bytree` = 0.9,
- `min_child_weight` = 3,

- `reg_lambda = 1.0`,
- `objective = reg:squarederror`,
- `tree_method = hist`.

We did not run a very large hyperparameter search because the dataset is already quite big. Instead, we used stable settings that give good performance without clear overfitting.

## 4.2 Time-Based Cross-Validation and Ensemble

Because the competition evaluates on future games, we use a time-based validation strategy. We split the data by week:

- **Fold 1:** train on weeks 1–12, validate on weeks 13–15,
- **Fold 2:** train on weeks 1–15, validate on weeks 16–17,
- **Fold 3:** train on weeks 1–17, validate on week 18.

For each fold, we train one model for  $\Delta x$  and one for  $\Delta y$ . At prediction time, we average the outputs from all fold models (an ensemble). This reduces variance and gives more stable predictions across different weeks.

The fold-wise validation RMSE values are:

Table 1: Time-based cross-validation RMSE for the XGBoost model.

| Fold | Validation weeks | 2D RMSE (yards) |
|------|------------------|-----------------|
| 1    | 13–15            | 1.12            |
| 2    | 16–17            | 1.24            |
| 3    | 18               | 1.04            |
| Mean | –                | 1.14            |

These results show that the model performs consistently on later weeks and does not strongly overfit to early games.

## 4.3 Kaggle Inference Setup

For the competition, we must run our model through an API that streams test data. We use the official `NFLInferenceServer` class provided by Kaggle. The server sends us batches of:

- `test`: rows to predict (game, play, player, frame),
- `test_input`: tracking data before the throw for those plays.

For each batch we:

1. build the same features as during training (normalization, history, geometry),
2. call all XGBoost models and average their predicted displacements,
3. reconstruct  $(x, y)$  and undo the left-right flip when needed,
4. return a DataFrame with the columns `x` and `y`.

This code runs inside the Kaggle environment and is evaluated on hidden games.

## 4.4 Performance and Leaderboard Result

On our time-based cross-validation, the XGBoost ensemble reaches a mean 2D RMSE of about 1.14 yards. On the Kaggle public leaderboard, our best submission with this model achieves a score of **0.773**, which corresponds to a rank of **790** at the time of submission.

## NFL Big Data Bowl 2026 - Prediction

Predict player movement while the ball is in the air



Overview Data Code Models Discussion **Leaderboard** Rules Team Submissions

### Leaderboard

Raw Data Refresh

alae

This leaderboard is calculated with all of the test data.

Prize Contenders

| #   | Team    | Members | Score | Entries | Last |
|-----|---------|---------|-------|---------|------|
| 790 | alaeeee |         | 0.773 | 8       | 9d   |

Your Best Entry!  
Your most recent submission scored 0.786, which is not an improvement of your previous score. Keep trying!

Figure 1: Kaggle public leaderboard snapshot for our team (rank 790, score 0.773).

Other models we tried (LightGBM, CatBoost, LSTM) gave higher errors in our validation setting and did not beat this XGBoost configuration on Kaggle. Therefore, we selected XGBoost with history and geometry features as our final model.

## 5 Other Tested Models

Although the main focus of this report is the XGBoost model, we also experimented with other approaches to better understand the problem and to verify that our feature engineering and validation pipeline were sound.

### 5.1 LightGBM Models

We trained LightGBM regressors for  $x$  and  $y$  using a compact feature set. The features focused on:

- current kinematic state (position, speed, acceleration, direction, velocity components),
- simple context (ball landing coordinates, absolute yardline),
- a frame offset that measures how far we are into the ball-in-air period.

Two configurations were explored:

- a *baseline* LightGBM with very few trees and limited tuning, used mainly to check that the pipeline was correct,
- a *stronger* LightGBM with around 1000 trees, subsampling (0.8) and a moderate learning rate (0.05).

On the time-based validation split, the stronger LightGBM reached a 2D RMSE of about **1.52** yards. This is more than five times better than a naive physics baseline that simply extrapolates constant velocity (around 8 yards), but still clearly behind the XGBoost ensemble. An alternative LightGBM configuration with weaker features and less tuning reached a much higher error (around 7.3 yards), which confirmed how sensitive gradient boosting is to good feature design and reasonable hyperparameters.

### 5.2 CatBoost Model

We also trained CatBoost models with two separate regressors for  $\Delta x$  and  $\Delta y$ . CatBoost is also a gradient boosting method, but it uses symmetric trees and ordered boosting, which can help with stability on noisy tabular data.

For CatBoost we deliberately used a simplified set of around 18 numerical features:

- kinematics: time  $t$ , speed  $s$ , acceleration  $a$ , and initial velocity components  $v_{x0}, v_{y0}$ ,
- ball and field context: distance and angle to the ball, time to ball, distances to sideline and goal line,
- simple crowding indicators and basic biometrics (height, weight, age).

Hyperparameters were chosen with a strong regularisation focus: very low learning rate (0.01), many potential iterations but with early stopping, depth around 8, and strong L2 regularisation. This “anti-overfitting” setup gave a stable 2D RMSE of about **1.51** yards on our validation split.

In summary, CatBoost confirmed that our physics-inspired features are useful and that different boosting algorithms can reach similar performance. However, it did not outperform the XGBoost ensemble and was slightly slower to tune, so we kept it as a strong reference model rather than the final choice.

### 5.3 LSTM Sequence Model

To explicitly model temporal dependencies, we implemented an LSTM sequence model. Instead of treating each frame independently, the LSTM receives a short sequence (for example, the last 10 frames before the throw) for each player and predicts the displacement after the throw.

Each time step in the sequence includes:

- $x, y$  coordinates,
- speed and acceleration,
- velocity components,
- angles (direction, orientation) in radians,
- simple ball-relative features such as distance and angle to the landing point.

The network consists of stacked LSTM layers with dropout in between, followed by a dense layer that outputs  $\Delta x$  and  $\Delta y$ . We trained with the Adam optimizer and early stopping on a held-out validation set.

The LSTM clearly beat the constant-velocity baseline and produced realistic, smooth trajectories, but the final 2D RMSE (around **2.5** yards) was still worse than the gradient-boosted models. Training also required significantly more time and effort to tune.

### 5.4 Summary of Model Performance

Table 2 summarises the approximate 2D RMSEs on our internal validation splits.

Table 2: Approximate 2D RMSE on validation data for different models.

| Model                      | 2D RMSE (yards) | Notes                        |
|----------------------------|-----------------|------------------------------|
| Constant-velocity baseline | $\approx 8.0$   | Simple physics extrapolation |
| LSTM sequence model        | $\approx 2.5$   | Uses 10-frame sequences      |
| LightGBM (tuned)           | $\approx 1.52$  | Compact feature set          |
| CatBoost (tuned)           | $\approx 1.51$  | Simplified 18 features       |
| XGBoost ensemble           | $\approx 1.14$  | History + geometry features  |

These experiments support the fact that for structured tabular problems, good feature engineering combined with well-tuned gradient boosting can outperform more complex deep models, especially when training time and data are limited.



## 6 Discussion

Looking back at our work, several design choices were especially important and align with core machine learning principles.

**Target choice and inductive bias.** By predicting displacement instead of absolute coordinates, we built an inductive bias into the model: it learns “how players move” rather than “where they stand”. This reduces variance and makes the task easier to learn, because the same movement pattern can be reused across different parts of the field. In practice, this was more effective than trying to predict  $(x, y)$  directly.

**Feature engineering vs. model complexity.** We invested a lot of effort into feature engineering: history summaries, ball-relative geometry, distances to sidelines and endzones, and simple physics-based baselines. The comparison between models shows that these features are responsible for most of the performance gains. Once the features were rich enough, several boosting methods (XGBoost, LightGBM, CatBoost) all reached strong performance, and adding a more complex LSTM did not automatically win. This illustrates that, for tabular data, careful feature design can be more important than choosing the most sophisticated model.

**Temporal validation and data leakage.** Because the leaderboard evaluates on future games, a naive random train/validation split would leak future information into the training set and give overly optimistic results. By splitting by week and always validating on later weeks, we respected the time structure of the data. The fact that the cross-validation RMSE (around 1.14) and the Kaggle score (0.773) were reasonably aligned is a sign that our validation strategy reduced data leakage and improved the reliability of our estimates.

**Bias–variance trade-off and ensembling.** We used an ensemble of three XGBoost models trained on different time ranges. Averaging their predictions reduces variance and makes the model more robust to particular weeks or unusual plays. This is a simple but effective application of the bias–variance trade-off: each individual model might slightly overfit its training weeks, but the ensemble smooths out idiosyncrasies without increasing bias too much.

## 7 Conclusion

In this project we built a complete machine learning pipeline for the NFL Big Data Bowl 2026 competition. Starting from raw tracking data, we:

- cleaned and normalized the plays and made the field direction consistent,
- engineered features based on physics, player history, ball context, and field geometry,
- trained and compared several models (constant-velocity baselines, LightGBM, CatBoost, LSTM, XGBoost),
- selected an XGBoost ensemble as our final approach and integrated it into the Kaggle inference API.

The XGBoost model with history and geometric features, trained with a time-based cross-validation scheme, achieved a strong validation RMSE and a public Kaggle score of 0.773 with a rank of 790. This project gave us useful insights that more complex models are not always better when the data is structured and tabular and properly handling time and avoiding leakage is critical in forecasting tasks. Also, ensembling and regularisation help control variance and improve robustness,

## 8 Acknowledgement

The software used for data processing, modelling, and visualization in this project is Python (version 3.12.2) [1]. All code files and the list of installed libraries can be found in the README of our GitHub repository.

For language quality, all written materials were checked for grammar and clarity using Grammarly [2]. Claude and Gemini assisted with troubleshooting and debugging parts of the Python code [3, 4]. All suggestions from these tools were critically reviewed and validated by us, in line with the guidelines of UHasselt on the responsible use of generative AI in education and assessment [5]. The selection and use of generative AI tools in this project followed the recommendations described in the UHasselt AI Tools documentation [5].

## Appendix

All code used in this project (data loading, feature engineering, model training, and Kaggle submission scripts) is available in a public GitHub repository:

- **Repository:** <https://github.com/3aLae/machine-learning-project>

## References

- [1] Python Software Foundation. *Python 3.12.2 Documentation*. <https://docs.python.org/release/3.12.2/>
- [2] Grammarly Inc. *Grammarly Writing Assistant*. <https://www.grammarly.com/>
- [3] Anthropic PBC. *Claude AI*. <https://claude.ai/>
- [4] Google LLC. *Gemini*. <https://gemini.google.com/>
- [5] Hasselt University. *Guidelines on the Responsible Use of Generative AI in Education and Assessment*. Retrieved from <https://www.uhasselt.be/en/aparte-sites-uhasselt-en/ai-hasselt-university/ai-tools>